UD04 – Introducción a ORM

Aplication.properties

El fichero “application.properties” en Spring es un archivo de configuración centralizado utilizado para definir propiedades y configuraciones para una aplicación Spring Boot. Este archivo permite a los desarrolladores personalizar el comportamiento de su aplicación, incluyendo la configuración del servidor, parámetros de base de datos, niveles de logging, y más. Spring Boot utiliza estos valores para configurar automáticamente la aplicación durante el arranque.

Ubicación

Por defecto, el archivo “application.properties” se ubica en el directorio “src/main/resources” de un proyecto Spring Boot. Esta ubicación permite que el archivo sea empaquetado dentro del artefacto construido (por ejemplo, un archivo JAR o WAR), haciendo que las configuraciones estén disponibles en el entorno de ejecución.

Funcionalidades Clave

- Configuración de Base de Datos: Permite especificar detalles de conexión a la base de datos, como URL, nombre de usuario, contraseña, y configuraciones de pool de conexiones.

```
spring.datasource.url=jdbc:mysql://localhost:3306/mi_base_de_datos
spring.datasource.username=usuario
spring.datasource.password=contraseña
```

- Configuración del Servidor: Define propiedades del servidor embebido, como el puerto en el que se ejecuta la aplicación o el contexto de la aplicación.

```
server.port=8080
server.servlet.context-path=/miapp
```

- Configuración de Logging: Personaliza los niveles de logging de la aplicación y de las dependencias.

```
logging.level.org.springframework=INFO
logging.level.com.miapp=DEBUG
```

- Configuración de JPA/Hibernate: Configura aspectos de JPA y Hibernate, como el dialecto de la base de datos o la generación de DDL.

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

- Configuraciones Específicas de la Aplicación: Además de las configuraciones predeterminadas de Spring Boot, los desarrolladores pueden definir sus propias propiedades personalizadas para ser utilizadas dentro de la aplicación.

```
miapp.configuracion.especial=valorEspecial
```

Ventajas

- Centralización de Configuraciones: Facilita la gestión de configuraciones al mantenerlas en un único lugar.

- Flexibilidad: Permite sobrescribir configuraciones para diferentes entornos (desarrollo, prueba, producción) utilizando perfiles de Spring ("application-{profile}.properties").

- Automatización: Spring Boot utiliza estas configuraciones para autoconfigurar componentes de la aplicación, reduciendo la necesidad de configuración manual y código boilerplate.

El archivo "application.properties" ayuda a los desarrolladores a gestionar las configuraciones de su aplicación de manera eficiente y flexible, adaptándose a las necesidades específicas de cada proyecto.

Ejemplo MySQL:

```
spring.datasource.url=jdbc:mysql://localhost:3306/appPeliculas
spring.datasource.username=appPeliculas
spring.datasource.password=1234
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

H2

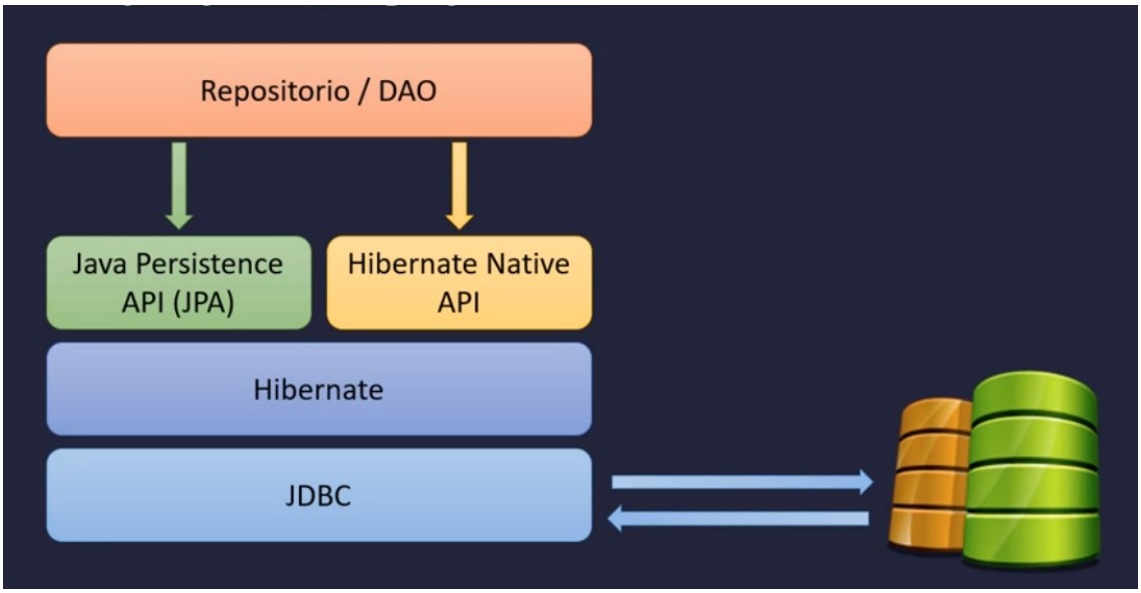
```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.defer-datasource-initialization=true
spring.jpa.show-sql=true
spring.h2.console.enabled=true
```

Datos de ejemplo:

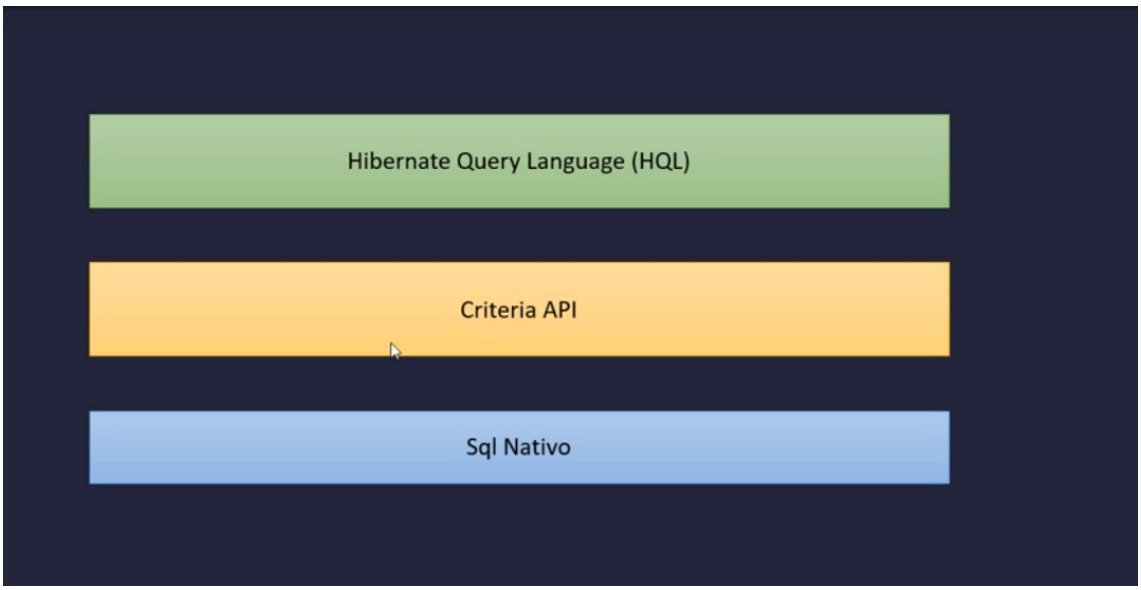
Crear un archivo import.sql en resources, al arrancar hace el insert, siempre que `spring.jpa.hibernate.ddl-auto=create`

Hibernate

Es una herramienta de mapeo de objeto relacional (ORM) que permite trabajar los datos de una base de datos (RDBMS) en forma de clases y objetos (lenguaje POO).



Tipos de consultas:



HQL:

Consultas sobre las entities:

Select c from cliente c;

Criteria:

(No visto en clase.)

Permite construir SQL de forma dinámica, de forma programática.

```
CriteriaQuery<Cliente> criteria = builder.createQuery( Cliente.class
);
Root<Cliente> root = criteria.from( Cliente.class );
criteria.select( root );
criteria.where( builder.equal( root.get( Person_nombre ), "John Doe" )
);
```

Introducción Spring Data JPA

JPA es el estándar, conjunto de interfaces y clases abstractas, que especifica cómo se debe implementar el api de persistencia de Java.

Después tenemos las implementaciones al estándar JPA, por ejemplo hibernate, que implementa y provee el estándar JPA.

ORM → Objetos mapeados a tablas.

https://hibernate.org/

orm

https://hibernate.org/orm/

punto 15-16

https://spring.io/projects/spring-data

Spring data JPA

https://spring.io/projects/spring-data-jpa

learn

https://spring.io/projects/spring-data-jpa#learn

Doc reference y api

https://docs.spring.io/spring-data/jpa/reference/

→JPA → Core concepts

https://docs.spring.io/spring-data/jpa/reference/repositories/core-concepts.html

Diferencia entre crudrepository y JPArepository. → JPArepository incluye métodos para ordenar e iterar. → PagingAndSortingRepository

→JPA Query Methods

https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html

Vemos muchos predicados, o palabras clave de las consultas.

→En la derecha → Using @Query y Native Queries

Anotaciones Básicas de JPA para la Creación de una Entidad

Java Persistence API (JPA) y su implementación popular, Hibernate, utilizan anotaciones para mapear clases de Java a tablas de bases de datos, facilitando la persistencia de datos en aplicaciones Java. Estas anotaciones permiten definir cómo se almacenan y recuperan los objetos Java de la base de datos sin necesidad de escribir consultas SQL explícitas. A continuación, se describen las anotaciones básicas y algunas importantes para la creación de una entidad en JPA:

- "@Entity": Indica que una clase es una entidad JPA. Esta anotación se coloca sobre la declaración de clase para informar a JPA que la clase debe ser mapeada a una tabla en la base de datos.

- "@Table": Especifica la tabla en la base de datos con la que se debe mapear la entidad. Aunque es opcional (si no se especifica, se utiliza el nombre de la clase como nombre de la tabla), es útil para configurar el nombre de la tabla y otros detalles de mapeo de tabla, como el esquema o las restricciones únicas.

```
@Entity
@Table(name = "mi_tabla")
public class MiEntidad { ... }
```

- "@Id": Marca el campo que actúa como la clave primaria de la entidad. Cada entidad debe tener un campo anotado con "@Id" para identificar de manera única cada instancia de la entidad en la base de datos.

- "@Column": Se utiliza para especificar el mapeo entre el campo de la entidad y la columna de la base de datos. Permite personalizar el nombre de la columna, su longitud, si puede ser nulo, y otras propiedades de la columna.

```
@Column(name = "nombre", nullable = false, length = 50)
private String nombre;
```

- “@GeneratedValue”: Especifica la estrategia de generación de valores para la clave primaria. Se utiliza junto con “@Id” para indicar cómo se generan los valores de identificación únicos (por ejemplo, autoincremento, secuencia, etc.).

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

- “@Enumerated”: Se utiliza para mapear un campo enum a una columna de la base de datos. Permite especificar si el mapeo debe usar el nombre del enum (String) o su ordinal (número).

- @Temporal: Se utiliza para mapear campos de fecha y hora (“java.util.Date” o “java.util.Calendar”) a una columna de base de datos, especificando el tipo de fecha/hora (fecha, hora, o fecha y hora).

Clase entidad:

```
@Entity
public class Cliente {

@Id
private Long id;
private String nombre;
private String apellido;
@Column(name="forma_pago")
private Date formaPago;
//Getters y setters
```

Si la anotación `@Entity` no tiene la propiedad table se une a una tabla con el mismo nombre “Cliente”.

La anotación `@id` para crear una clave primaria, podríamos cambiarle el nombre con `@Column`

Si no anotamos los atributos se crearán de manera básica.

Entity

POJO.

@Id

@GeneratedValue(strategy = GenerationType.Identity)

GenerationType.Identity vs GenerationType.Secuence

CamelCase pero en BBDD snakeCase

@Column (name = “programing_language”)

Ciclo de vida de las entidades.

@PrePersist

La anotación “@PrePersist” en Spring se utiliza en el contexto de JPA (Java Persistence API), que es un estándar para la persistencia de datos y la creación de ORM (Object-Relational Mapping) en Java.

“@PrePersist” se utiliza para configurar un método de devolución de llamada que se ejecutará justo antes de que la entidad correspondiente se inserte por primera vez en la base de datos. Es útil para realizar ciertas operaciones justo antes de la persistencia de la entidad, como establecer valores predeterminados o registrar la fecha y hora de creación.

Veamos un ejemplo de cómo se puede utilizar:

```
import jakarta.persistence.PrePersist;
import java.time.LocalDateTime;

@Entity
public class User {
    // ...

    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
    }

    // ...
}
```

En este ejemplo, el método “prePersist” se ejecutará justo antes de que se persista una nueva entidad “User”. Establece el campo “createdAt” en la fecha y hora actuales. De esta manera, cada vez que crees un nuevo “User”, su fecha de creación se registrará automáticamente.

@PostPersist

La anotación "@PostPersist" en Spring se utiliza en el contexto de JPA (Java Persistence API). Al igual que "@PrePersist", "@PostPersist" se utiliza para configurar un método de devolución de llamada, pero este se ejecuta justo después de que la entidad correspondiente se haya insertado con éxito en la base de datos.

Esto puede ser útil para realizar ciertas operaciones que dependen de la creación exitosa de la entidad, como el registro de eventos o la notificación a otros componentes de la aplicación.

Veamos un ejemplo de cómo se puede utilizar:

```
import jakarta.persistence.PostPersist;

@Entity
public class User {
    // ...

    @PostPersist
    public void postPersist() {
        System.out.println("User has been created with id: " +
this.id);
    }

    // ...
}
```

En este ejemplo, el método "postPersist" se ejecutará justo después de que se persista una nueva entidad "User". Imprime un mensaje indicando que el usuario ha sido creado, incluyendo su id. Esto podría ser útil para fines de depuración o registro.

Nota: Existen otras anotaciones como `@PreUpdate`, `@PostUpdate`

@Embedded

La anotación "@Embedded" en Spring se utiliza en el contexto de JPA (Java Persistence API) para indicar que una entidad tiene un atributo que es una clase incrustada.

Una clase incrustada es una clase que no es una entidad por sí misma, pero cuyos atributos se mapean directamente a la tabla de la entidad que la contiene. Esto puede ser útil para agrupar atributos relacionados juntos en su propia clase, lo que puede hacer que tu código sea más organizado y fácil de entender.

Veamos un ejemplo de cómo se puede utilizar:

```java
import jakarta.persistence.Embeddable;
import jakarta.persistence.Embedded;

@Entity
public class User {
    // ...

    @Embedded
    private Address address;

    // ...
}

@Embeddable
public class Address {
    private String street;
    private String city;
    private String state;
    private String zip;

    // getters and setters
}
```

En este ejemplo, “User” tiene un atributo “address” que es una instancia de la clase “Address”. “Address” está anotada con “@Embeddable” para indicar que es una clase incrustada, y el atributo “address” en “User” está anotado con “@Embedded” para indicar que es un atributo incrustado.

Como resultado, los atributos de “Address” (street, city, state, zip) se mapearán directamente a la tabla “User” en la base de datos.

Devolver solo atributos de un objeto.

```java
    @Query("select p.name, p.programmingLanguage from Person p")
    List<Object[]> obtenerPersonData();
```

Podemos envolver una consulta con Optional:

```java
    @Query("select p from Person p where p.id=?1")
    Optional<Person> findOne(Long id);


    @Query("select p from Person p where p.name=?1")
    Optional<Person> findOneName(String name);
```

Like:

```java
      Optional<Person> findByNameContaining(String name);
```

```
@Query("select p from Person p where p.name like %?1%")
Optional<Person> findOneLikeName(String name);
```

Save:

repositorio.save(objeto)

@transactional

La anotación “@Transactional” en Spring se utiliza para indicar que un método o clase debe tener un comportamiento transaccional.

Una transacción en el contexto de una base de datos es una secuencia de acciones que se ejecutan como una única unidad de trabajo. Si todas las acciones se completan con éxito, los cambios en la base de datos se confirman (o "commitean"). Si alguna de las acciones falla, todos los cambios se deshacen (o "rollbackean").

Cuando marcas un método con “@Transactional”, Spring asegura que este método se ejecute dentro de una transacción. Si el método completa su ejecución con éxito, Spring confirmará la transacción automáticamente. Si ocurre una excepción no controlada, Spring deshará la transacción.

Por ejemplo:

```
@Service
public class UserService {
    private final UserRepository userRepository;

    @Autowired
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Transactional
    public void updateUser(Long id, String newName) {
        User user = userRepository.findById(id);
        user.setName(newName);
        userRepository.save(user);
    }
}
```

En este ejemplo, el método “updateUser” se ejecuta dentro de una transacción. Si “userRepository.findById” o “userRepository.save” lanzan una excepción, Spring deshará cualquier cambio en la base de datos que haya ocurrido dentro de “updateUser”.

La anotación “@Transactional” puede hacer que tu código sea más fácil de entender y puede ayudar a prevenir errores de consistencia en tu base de datos.

Consultas:

@Transactional (readOnly= true)

JPQL:

Podemos obtener los datos tipados de la BBDD:

```
    @Query("select p.name from Person p where p.id=?1")
    String getNameById(Long id);

    @Query("select p.id from Person p where p.id=?1")
    Long getIdById(Long id);

    @Query("select CONCAT(p.name, ' ', p.lastname) as fullname from
Person p where p.id=?1")
    String getFullNameById(Long id);
```

Asociaciones:

Cuando hablamos de asociaciones, debemos destacar que existen 2 tipos de asociaciones, unidireccionales, y bidireccionales, que se basa en la dirección de la asociación.

- @OneToOne
- @ManyToOne
- @OneToMany
- @OneToOne

@OneToOne

Para explicar “@OneToOne” sin bidireccionalidad, consideremos un ejemplo simple donde cada usuario tiene asignado exactamente un perfil. En este caso, no necesitamos navegar desde “Perfil” a “Usuario”, solo de “Usuario” a “Perfil”.

Veamos un ejemplo de Relación “@OneToOne”

```
@Entity
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "perfil_id", referencedColumnName = "id")
```

```
    private Perfil perfil;

    // Getters y setters...
}

@Entity
public class Perfil {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String detalles;

    // Getters y setters...
}
```

En este ejemplo:

- La entidad “Usuario” tiene un campo “perfil” que es una referencia a la entidad “Perfil”. La anotación “@OneToOne” indica que se trata de una relación uno a uno. La anotación “@JoinColumn” especifica la columna que se utilizará como clave foránea (“perfil_id”) para establecer la relación con “Perfil”.

- La entidad “Perfil” no tiene referencia a “Usuario”, lo que significa que la relación es unidireccional. Solo puedes navegar de “Usuario” a “Perfil”, pero no al revés.

Este modelo es adecuado cuando tienes una clara relación uno a uno entre dos entidades, pero solo necesitas acceder a una de las entidades desde la otra, no necesitando la navegación inversa.

@ManyToOne

La anotación “@ManyToOne” en Spring JPA se utiliza para establecer una relación de muchos a uno entre dos entidades.

En una relación de muchos a uno, muchas entidades de un tipo pueden estar asociadas con una única entidad de otro tipo. Por ejemplo, podrías tener una entidad “Order” y una entidad “Customer”, donde cada “Order” está asociada a un “Customer”, pero un “Customer” puede tener muchas “Order”.

Veamos un ejemplo de cómo se puede utilizar:

```
import jakarta.persistence.ManyToOne;

@Entity
public class Order {
    // ...

    @ManyToOne
    private Customer customer;

    // ...
```

```
}

@Entity
public class Customer {
    // ...
}
```

En este ejemplo, cada “Order” está asociada a un “Customer”. Cuando se guarda una “Order”, se guarda una referencia al “Customer” asociado en la base de datos.

Puedes usar “@ManyToOne” junto con “@JoinColumn” para personalizar la columna que se utiliza para almacenar la referencia al “Customer”.

```
@ManyToOne
@JoinColumn(name="customer_id")
private Customer customer;
```

En este caso, la referencia al “Customer” se almacenará en una columna llamada “customer_id” en la tabla “Order”.

@OneToMany

(crea tabla intermedia, cuidado)

La anotación “@OneToMany” en Spring JPA se utiliza para establecer una relación de uno a muchos entre dos entidades.

En una relación de uno a muchos, una entidad de un tipo puede estar asociada con muchas entidades de otro tipo. Por ejemplo, podrías tener una entidad “Customer” y una entidad “Order”, donde un “Customer” puede tener muchas “Order”, pero cada “Order” está asociada a un solo “Customer”.

Veamos un ejemplo de cómo se puede utilizar:

```
import jakarta.persistence.OneToMany;

@Entity
public class Customer {
    // ...

    @OneToMany
    private List<Order> orders;

    // ...
}

@Entity
public class Order {
    // ...



    // ...
}
```

En este ejemplo, cada “Customer” puede tener muchas “Order”. La anotación “@OneToMany” en el campo “orders” en “Customer” establece esta relación. El atributo “mappedBy” indica el campo en la entidad “Order” que mantiene la relación inversa.

Cuando se guarda un “Customer” con una lista de “Order”, JPA actualizará la base de datos de manera que cada “Order” en la lista apunte a la “Customer” correspondiente.

@OneToMany sin tabla intermedia.

Cuando usamos “@OneToMany” junto con el atributo “mappedBy”, JPA no crea ninguna tabla intermedia. En su lugar, la relación se guarda en una columna de clave externa (foreign key) dentro de la tabla del lado “muchos”.

Siguiendo el ejemplo anterior, la tabla “orders” tendrá una columna “customer_id” que apunta al “Customer” al que pertenece cada pedido. Para ello, la entidad “Order” debe tener el campo que mantiene la relación, con la anotación “@ManyToOne”:

```
@Entity
public class Customer {
    // ...

    @OneToMany
    @JoinColumn(name = "customer_id")
    private List<Order> orders;

    // ...
}
```

Con “@JoinColumn”, JPA no crea la tabla intermedia, sino que añade la columna “customer_id” directamente en la tabla “orders”. La relación sigue siendo unidireccional, porque “Order” sigue sin tener un campo “customer” en su clase.

@ManyToMany

La anotación “@ManyToMany” en JPA también se puede utilizar para definir relaciones de muchos a muchos sin bidireccionalidad, es decir, cuando solo una de las entidades necesita conocer la existencia de la otra. Esto puede ser útil para simplificar el modelo de dominio cuando no necesitas navegar la relación en ambas direcciones.

Veamos un ejemplo de una relación “@ManyToMany” sin bidireccionalidad:

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // Otros atributos...
```

```
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();

    // Getters y setters...
}

@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // Otros atributos...

    // No hay referencia a Student desde Course

    // Getters y setters...
}
```

En este ejemplo, la entidad “Student” tiene una relación “@ManyToMany” con “Course”, pero la entidad “Course” no tiene conocimiento de “Student”. La relación se configura de la misma manera que en el ejemplo bidireccional, utilizando una tabla de unión especificada por la anotación “@JoinTable”. Sin embargo, en este caso, no se utiliza “mappedBy” en la entidad “Course” porque no estamos estableciendo una relación bidireccional.

Este enfoque es útil cuando solo necesitas acceder a los cursos de un estudiante, pero no necesitas acceder a los estudiantes de un curso directamente desde la entidad “Course”. Reducir la bidireccionalidad en las relaciones puede simplificar el modelo de dominio y mejorar el rendimiento al evitar cargas innecesarias de datos relacionados.

Cascade

En JPA, la propiedad “cascade” se utiliza para propagar las operaciones de la entidad padre a las entidades hijo. Esto es útil en las relaciones entre entidades, como “@OneToOne”, “@OneToMany”, “@ManyToOne” y “@ManyToMany”.

Las operaciones en cascada disponibles son:

- “CascadeType.PERSIST”: Si la entidad padre se persiste, la entidad hijo también se persistirá.
- “CascadeType.REMOVE”: Si la entidad padre se elimina, la entidad hijo también se eliminará.
- “CascadeType.MERGE”: Si la entidad padre se fusiona, la entidad hijo también se fusionará.
- “CascadeType.REFRESH”: Si la entidad padre se actualiza, la entidad hijo también se actualizará.

- “CascadeType.DETACH”: Si la entidad padre se desconecta, la entidad hijo también se desconectará.
- “CascadeType.ALL”: Todas las operaciones anteriores se aplicarán.

Veamos un ejemplo de cómo se puede utilizar:

```
@Entity
public class Post {
    // ...

    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL)
    private List<Comment> comments = new ArrayList<>();

    // ...
}

@Entity
public class Comment {
    // ...

    @ManyToOne
    private Post post;

    // ...
}
```

En este ejemplo, cada “Post” puede tener muchos “Comment”. La anotación “@OneToMany” en el campo “comments” en “Post” establece esta relación. La propiedad “cascade = CascadeType.ALL” indica que cualquier operación realizada en “Post” se propagará a sus “Comment” asociados. Por ejemplo, si un “Post” se elimina, todos sus “Comment” asociados también se eliminarán.

orphanRemoval

La propiedad “orphanRemoval” en JPA se utiliza para especificar que las entidades hijo que ya no están asociadas a la entidad padre deben eliminarse. Es útil en las relaciones “@OneToOne” y “@OneToMany”.

Cuando “orphanRemoval” está establecido en “true”, si quitas la relación entre la entidad padre y la entidad hijo (por ejemplo, quitas un objeto de una colección en la entidad padre), la entidad hijo se eliminará automáticamente de la base de datos. Esto es útil cuando la entidad hijo no tiene sentido sin la entidad padre.

Veamos un ejemplo de cómo se puede utilizar:

```
@Entity
public class Cart {
    // ...

    @OneToMany(mappedBy = "cart", orphanRemoval = true)
    private List<Item> items = new ArrayList<>();
```

```
    // ...
}

@Entity
public class Item {
    // ...

    @ManyToOne
    private Cart cart;

    // ...
}
```

En este ejemplo, cada “Cart” puede tener muchos “Item”. La anotación “@OneToMany” en el campo “items” en “Cart” establece esta relación. La propiedad “orphanRemoval = true” indica que si un “Item” se quita de la “Cart”, se eliminará automáticamente de la base de datos. Esto tiene sentido en este contexto, ya que un “Item” que no está en una “Cart” ya no es relevante y puede ser eliminado.

Bidireccionalidad

La bidireccionalidad en Hibernate y Spring JPA se refiere a la capacidad de navegar una relación entre dos entidades en ambas direcciones. En una relación bidireccional, cada entidad tiene una referencia a la otra, permitiendo el acceso a los datos relacionados desde cualquiera de los dos lados de la relación.

Las relaciones bidireccionales pueden ser de varios tipos, incluyendo “@OneToOne”, “@OneToMany”/”@ManyToOne”, y “@ManyToMany”. La bidireccionalidad se establece mediante el uso de anotaciones en ambos lados de la relación, con una de las entidades actuando como el lado "propietario" de la relación (el que maneja la actualización de la relación en la base de datos) y la otra como el lado "inverso" o "mapeado por".

Ejemplo de “@OneToMany” y “@ManyToOne” (Relación Bidireccional)

Considera una relación entre “Post” y “Comment” donde un “Post” puede tener muchos “Comment”, pero un “Comment” pertenece a un solo “Post”.

```
@Entity
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL,
orphanRemoval = true)
    private List<Comment> comments = new ArrayList<>();
```

```
    // Getters y setters...
}

@Entity
public class Comment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "post_id")
    private Post post;

    // Getters y setters...
}
```

- **Lado Propietario (“Comment”)**: El lado propietario es donde defines la columna de clave foránea (“post_id”). Aquí, “Comment” es el propietario de la relación porque contiene la columna de clave foránea que referencia a “Post”.

- **Lado Inverso (“Post”)**: El lado inverso es donde usas “mappedBy” para indicar que la relación es manejada por el otro lado. En este caso, “Post” es el lado inverso, indicando que la relación está mapeada por la propiedad “post” en “Comment”.

Beneficios de la Bidireccionalidad

- **Navegación**: Permite navegar la relación en ambas direcciones, desde “Post” a “Comment” y viceversa.

- **Sincronización**: Ayuda a mantener sincronizadas las dos partes de la relación, asegurando la integridad de los datos.

- **Eficiencia en Consultas**: Puede mejorar la eficiencia de las consultas al permitir el acceso directo a los datos relacionados desde cualquiera de los lados de la relación.

Consideraciones

- **Manejo de Lado Propietario**: Es importante manejar correctamente el lado propietario de la relación para asegurar que las actualizaciones en la base de datos reflejen el estado de los objetos en memoria.

- **Rendimiento**: Las relaciones bidireccionales pueden afectar el rendimiento si no se usan con cuidado, especialmente si se cargan grandes volúmenes de datos relacionados innecesariamente.

La bidireccionalidad es una buena herramienta en JPA y Hibernate, pero requiere un diseño cuidadoso y una gestión adecuada para aprovechar sus beneficios sin incurrir en problemas de rendimiento o mantenimiento.

Otro ejemplo con Bidireccionalidad.

Una relación “@OneToOne” en JPA (Java Persistence API) se utiliza para modelar una relación en la que una entidad está asociada con exactamente una instancia de otra entidad. Por ejemplo, considera una situación donde cada persona tiene asignado un único pasaporte, y cada pasaporte está asociado con una única persona. Esta es una relación uno a uno entre “Persona” y “Pasaporte”.

Para implementar una relación “@OneToOne” en JPA, puedes usar la anotación “@OneToOne”. Aquí hay un ejemplo que muestra cómo hacerlo:

Ejemplo de Relación “@OneToOne”

```
@Entity
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "pasaporte_id", referencedColumnName = "id")
    private Pasaporte pasaporte;

    // Getters y setters...
}

@Entity
public class Pasaporte {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String numero;

    @OneToOne(mappedBy = "pasaporte")
    private Persona persona;

    // Getters y setters...
}
```

En este ejemplo:

- La entidad “Persona” tiene un campo “pasaporte” que es una referencia a la entidad “Pasaporte”. La anotación “@OneToOne” indica que se trata de una relación uno a uno. La anotación “@JoinColumn” especifica la columna que se utilizará como clave foránea para establecer la relación.

- La entidad “Pasaporte” tiene un campo “persona” que referencia a la entidad “Persona”. La anotación “@OneToOne(mappedBy = "pasaporte")” indica que esta es la otra parte de una relación uno a uno bidireccional y que el mapeo es manejado por el campo “pasaporte” en la entidad “Persona”.

Consideraciones

- Bidireccionalidad: En el ejemplo, la relación es bidireccional, lo que significa que puedes navegar desde “Persona” a “Pasaporte” y viceversa. La bidireccionalidad se establece mediante el uso de “mappedBy” en el lado no propietario (“Pasaporte”).

- Cascada: La opción “cascade = CascadeType.ALL” en “Persona” indica que las operaciones como persistir, eliminar, etc., aplicadas a “Persona” también deben aplicarse a “Pasaporte”. Esto es útil para mantener sincronizadas las operaciones entre las entidades relacionadas.

- Clave Foránea: La relación “@OneToOne” generalmente implica una restricción de clave foránea en la base de datos, asegurando la integridad referencial entre las dos tablas.

Este modelo es útil cuando necesitas asegurar que cada entidad en una relación esté vinculada a una y solo una instancia de la otra entidad, como en el caso de personas y sus pasaportes.

ManyToMany Bidireccional

En una relación de muchos a muchos, una entidad de un tipo puede estar asociada con muchas entidades de otro tipo, y viceversa. Por ejemplo, un “Student” puede estar matriculado en muchos “Course”, y un “Course” puede tener muchos “Student”.

Este tipo de relación siempre necesita una tabla intermedia (“student_course”) que guarda las parejas de identificadores. Al ser bidireccional, ambas entidades tienen su lista y una de ellas es la propietaria de la relación: la que define la tabla intermedia con “@JoinTable”. La otra, la inversa, usa “mappedBy”.

```
import javax.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
public class Student {
    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();

    public void addCourse(Course course) {
        this.courses.add(course);
        course.getStudents().add(this);
    }

    public void removeCourse(Course course) {
```

```
        this.courses.remove(course);
        course.getStudents().remove(this);
    }

    // ...
}

@Entity
public class Course {
    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToMany(mappedBy = "courses", cascade = {CascadeType.PERSIST,
CascadeType.MERGE})
    private Set<Student> students = new HashSet<>();

    // ...
}
```

En este ejemplo, “Student” es el lado propietario porque tiene la anotación “@JoinTable”, y “Course” es el lado inverso porque usa “mappedBy”. Los métodos “addCourse” y “removeCourse” sirven para mantener sincronizados los dos lados de la relación, de modo que siempre se actualicen a la vez.

¿Qué hace cada tipo de Cascade?

“CascadeType.PERSIST”: al guardar una entidad nueva, se guardan también las entidades relacionadas que sean nuevas.

“CascadeType.MERGE”: al actualizar una entidad, los cambios se propagan a las entidades relacionadas.

¿Por qué no usar Cascade.ALL en el lado hijo?

“CascadeType.ALL” incluye todas las operaciones: PERSIST, MERGE, REMOVE, REFRESH y DETACH. En una relación de muchos a muchos, las entidades relacionadas se comparten con otras, y por eso es peligroso usarlo, sobre todo en el lado hijo. Si lo ponemos en ambos lados, pueden aparecer estos problemas:

Borrados en cadena (REMOVE): si borramos un “Student”, JPA borrará también todos sus “Course”. Pero esos cursos pueden tener otros alumnos matriculados, y al borrarse cada curso también se borrarían sus alumnos, y así sucesivamente. Un simple borrado puede acabar eliminando medio contenido de la base de datos.

Pérdida de datos inesperada: borrar un curso que otros alumnos siguen cursando hace que esos alumnos pierdan esa relación (o incluso que se borren ellos), sin que el programador lo haya pedido.

Errores de integridad: si el borrado en cadena choca con otras tablas que referencian a esas entidades, la base de datos lanzará errores de clave externa difíciles de depurar.

Peor rendimiento: al ser bidireccional, cada operación (sobre todo REFRESH, MERGE y DETACH) recorre las relaciones en los dos sentidos, cargando colecciones enteras de datos que quizá no necesitábamos.

Por eso, la recomendación es usar solo “PERSIST” y “MERGE” en el lado hijo (y evitar siempre “REMOVE”), y dejar que el borrado de las relaciones se haga de forma explícita con métodos como “removeCourse”.

Recuerda:

Usa “Set” en lugar de “List” en las relaciones ManyToMany. Con “List”, Hibernate borra todas las filas de la tabla intermedia y las vuelve a insertar cuando se modifica la colección, lo que es bastante ineficiente.

Fíjate en que he evitado “REMOVE” también en el lado propietario. En muchos a muchos, la buena práctica general es no propagar nunca el borrado, ni siquiera desde el lado padre, por las mismas razones que se explican arriba.

Recursión infinita en entidades bidireccionales.

Al utilizar los métodos “toString()” generados automáticamente por el IDE sin personalización adecuada, se produce una excepción que se conoce como "StackOverflowError" debido a la recursión infinita.

¿Cómo ocurre?

Cuando tienes dos entidades con una relación bidireccional (por ejemplo, “@OneToMany” y “@ManyToOne” entre “ClaseA” y “ClaseB”), y ambas entidades incluyen referencias a la otra en sus métodos “toString()”, intentar imprimir una instancia de cualquiera de estas entidades resultará en una llamada recursiva infinita entre estos métodos “toString()”. Esto sucede porque:

1.- Al intentar imprimir “ClaseA”, su método “toString()” intenta imprimir también la información de “ClaseB” asociada.
2.- Al imprimir “ClaseB”, su método “toString()” intenta imprimir la información de “ClaseA” asociada.
3.- Este ciclo continúa indefinidamente, ya que cada entidad intenta imprimir la otra, llevando eventualmente a un desbordamiento de la pila de llamadas (“StackOverflowError”).

Ejemplo:

```
class ClaseA {
    @OneToOne
    private ClaseB claseB;

    @Override
    public String toString() {
        return "ClaseA{" + "claseB=" + claseB + '}';
```

```
    }
}

class ClaseB {
    @OneToOne
    private ClaseA claseA;

    @Override
    public String toString() {
        return "ClaseB{" + "claseA=" + claseA + '}';
    }
}
```

Intentar imprimir una instancia de “ClaseA” o “ClaseB” resultará en un “StackOverflowError”.

Soluciones:

Para evitar este problema, puedes:

- Personalizar los métodos “toString()”: Modifica los métodos “toString()” para que no incluyan relaciones bidireccionales directamente, o maneja la impresión de manera condicional para evitar la recursión.

- Usar “@ToString.Exclude” con Lombok: Si estás utilizando Lombok, puedes excluir las relaciones bidireccionales en tus métodos “toString()” usando la anotación “@ToString.Exclude”.

- Implementar lógica de control: Añade lógica para controlar la profundidad de la recursión o para detectar y evitar la inclusión de referencias ya impresas.

Ejemplo solucionado:

```
@Entity
class ClaseA {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(mappedBy = "claseA")
    private ClaseB claseB;

    // Getters y Setters

    @Override
    public String toString() {
        // Implementación personalizada para evitar StackOverflowError
        return "ClaseA{" + "id=" + id + '}';
    }
}

@Entity
class ClaseB {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    private ClaseA claseA;
```

```
    // Getters y Setters

    @Override
    public String toString() {
        // Implementación personalizada para evitar StackOverflowError
        return "ClaseB{" + "id=" + id + '}';
    }
}
```

Evitar la recursión infinita en los métodos “toString()” es crucial para prevenir “StackOverflowError” y asegurar que tu aplicación maneje correctamente las relaciones bidireccionales.